

Institute of Engineering, Purwanchal Campus
Department of Electronics, Communication and Information Engineering

Assignment 01

Hexagonal Cellular Cluster Allocation
Co-channel Reuse Distance Proof

Submitted By

Name Suwarna Pyakurel
Roll No. PUR079BEI043
Email suwarna.079bei@ioepc.edu.np
Faculty Electronics, Communication and Information Engineering
Level Fourth Year / First Part
Subject Wireless Communication Systems

July 1, 2026

Contents

1	Introduction	1
2	Hexagonal Cluster Allocation Program	2
2.1	Problem Statement	2
2.2	Methodology	2
2.2.1	Coordinate Conversion	2
2.2.2	Cluster Assignment	2
2.2.3	Hexagon Placement	3
2.3	Source Code: <code>hexagonal_cluster.py</code>	3
2.4	Program Output	8
2.4.1	Case 1: $i = 1, j = 1, N = 3$	8
2.4.2	Case 2: $i = 2, j = 1, N = 7$	9
2.4.3	Case 3: $i = 3, j = 2, N = 19$	10
2.5	Sample Console Output	10
3	Proof: Co-channel Reuse Distance in Hexagonal Geometry	11
3.1	Statement	11
3.2	Geometric Setup	11
3.3	Derivation	12
3.3.1	Step 1: Law of Cosines	12
3.3.2	Step 2: Relation Between R_p and R	12
3.3.3	Step 3: Cluster Size	13
3.4	Numerical Verification ($i = 2, j = 1, R = 1$)	13
3.5	Supporting Code: <code>hexagonal_proof.py</code>	13
4	Conclusion	21

Chapter 1

Introduction

This report presents the complete solution for Assignment 01 in Wireless Communication Systems. The assignment consists of two related tasks:

1. Write a Python program that divides a workspace into regular hexagons and allocates cluster patterns using the formula

$$N = i^2 + ij + j^2 \tag{1.1}$$

where i and j are non-negative integers defining the frequency reuse vector.

2. Prove that for hexagonal geometry, the co-channel reuse distance is given by

$$D = R\sqrt{3N} \tag{1.2}$$

where R is the radius of a regular hexagonal cell and N is the cluster size.

The implementation uses `hexagonal_cluster.py` for cluster visualization and `hexagonal_proof.py` for geometric verification of the reuse distance formula.

Chapter 2

Hexagonal Cluster Allocation Program

2.1 Problem Statement

Given reuse parameters i and j , the program must:

- tessellate a workspace into regular hexagonal cells;
- compute cluster size $N = i^2 + ij + j^2$;
- assign a unique cluster ID to each cell using modular arithmetic on axial coordinates;
- display the resulting frequency reuse pattern graphically.

2.2 Methodology

2.2.1 Coordinate Conversion

Each cell in the offset grid is converted to axial coordinates (q, r) using the odd- q vertical layout:

$$q = \text{col} \tag{2.1}$$

$$r = \text{row} - \frac{\text{col} - (\text{col} \bmod 2)}{2} \tag{2.2}$$

2.2.2 Cluster Assignment

The cluster ID for a cell at (q, r) is computed as:

$$c = (q(i + j) - rj) \bmod N \tag{2.3}$$

$$d = (-qj + ri) \bmod N \tag{2.4}$$

$$\text{cluster} = (c + d) \bmod N \tag{2.5}$$

Each cell is labeled with cluster number $(\text{cluster} + 1)$ for display purposes.

2.2.3 Hexagon Placement

For a hexagon of radius R , the center of cell (col, row) is placed at:

$$x = 1.5R \cdot \text{col} \quad (2.6)$$

$$y = \sqrt{3} R \left(\text{row} + \frac{1}{2}(\text{col} \bmod 2) \right) \quad (2.7)$$

2.3 Source Code: hexagonal_cluster.py

```

1
2 import argparse
3 import math
4 import sys
5
6 import matplotlib.pyplot as plt
7 import numpy as np
8 from matplotlib.patches import RegularPolygon
9
10
11 def hex_to_axial(col: int, row: int) -> tuple[int, int]:
12     """Convert odd-q vertical offset coordinates to axial (q, r)."""
13     q = col
14     r = row - (col - (col & 1)) // 2
15     return q, r
16
17
18 def get_cluster_id(q: int, r: int, i: int, j: int, n: int) -> int:
19     """Assign a cluster index in [0, N-1] using the (i, j) reuse
20         vector."""
21     c = (q * (i + j) - r * j) % n
22     d = (-q * j + r * i) % n
23     return (c + d) % n
24
25
26 def cluster_size(i: int, j: int) -> int:
27     return i * i + i * j + j * j
28
29
30 def reuse_distance(radius: float, n: int) -> float:
31     """Co-channel reuse distance: D = sqrt(3N)."""
32     return radius * np.sqrt(3 * n)
33
34
35 def build_hex_grid(
36     i: int,
37     j: int,
38     rows: int = 12,
39     cols: int = 12,
40     radius: float = 1.0,
41 ):

```

```

41     """Return hex centers and cluster IDs for each cell in the grid.
42     """
43     n = cluster_size(i, j)
44     cells = []
45
46     for col in range(cols):
47         for row in range(rows):
48             x = col * 1.5 * radius
49             y = np.sqrt(3) * radius * (row + 0.5 * (col % 2))
50             q, r = hex_to_axial(col, row)
51             cluster = get_cluster_id(q, r, i, j, n)
52             cells.append(
53                 {
54                     "col": col,
55                     "row": row,
56                     "x": x,
57                     "y": y,
58                     "q": q,
59                     "r": r,
60                     "cluster": cluster,
61                 }
62             )
63
64     return cells, n
65
66 def plot_hexagonal_layout(
67     i: int,
68     j: int,
69     rows: int = 12,
70     cols: int = 12,
71     radius: float = 1.0,
72     output_path: str | None = None,
73     show: bool = True,
74 ) -> None:
75     n = cluster_size(i, j)
76     d = reuse_distance(radius, n)
77     q_ratio = d / radius
78
79     print(f"Cluster parameters: i = {i}, j = {j}")
80     print(f"Cluster size (N)      =  $i^2 + ij + j^2 = {i}^2 + {i} \times {j} + {j}^2 = {n}$ ")
81     print(f"Cell radius (R)          = {radius}")
82     print(f"Reuse distance (D)         =  $\sqrt{3N} = {d:.4f}$ ")
83     print(f"Reuse ratio (Q=D/R)       =  $\sqrt{3N} = {q_ratio:.4f}$ ")
84
85     cells, _ = build_hex_grid(i, j, rows, cols, radius)
86
87     # Distinct color per cluster (1-based labels in the plot).
88     cmap = plt.colormaps["tab20"].resampled(max(n, 1))
89

```

```

90     fig, ax = plt.subplots(figsize=(12, 11))
91
92     for cell in cells:
93         cluster = cell["cluster"]
94         color = cmap(cluster % cmap.N)
95
96         hexagon = RegularPolygon(
97             (cell["x"], cell["y"]),
98             numVertices=6,
99             radius=radius,
100             orientation=np.radians(30),
101             edgecolor="black",
102             linewidth=1.0,
103             facecolor=color,
104             alpha=0.85,
105         )
106         ax.add_patch(hexagon)
107         ax.text(
108             cell["x"],
109             cell["y"],
110             str(cluster + 1),
111             ha="center",
112             va="center",
113             fontsize=8,
114             fontweight="bold",
115             color="black",
116         )
117
118     ax.set_aspect("equal")
119     ax.autoscale_view()
120     ax.set_title(
121         f"Hexagonal Cellular Layout -  $N = i^2 + ij + j^2 = "$ 
122         f" $\{i\}^2 + \{i\} \times \{j\} + \{j\}^2 = \{n\} \backslash n$ "
123         f" $D = \sqrt{R(3N)} = \{radius\} \times \sqrt{(3 \times \{n\})} = \{d:.3f\}$ ",
124         fontsize=13,
125     )
126     plt.axis("off")
127     plt.tight_layout()
128
129     if output_path:
130         fig.savefig(output_path, dpi=150, bbox_inches="tight")
131         print(f"Saved plot to: {output_path}")
132
133     if show:
134         plt.show()
135     else:
136         plt.close(fig)
137
138
139 def parse_args(argv: list[str] | None = None) -> argparse.Namespace:
140     parser = argparse.ArgumentParser(

```

```

141     description="Hexagonal workspace with cluster allocation ( $N$ 
142         =  $i^2 + ij + j^2$ ).")
143 )
144 parser.add_argument("-i", "--i-param", type=int, dest="i", help=
145     "Reuse parameter i")
146 parser.add_argument("-j", "--j-param", type=int, dest="j", help=
147     "Reuse parameter j")
148 parser.add_argument("--rows", type=int, default=12, help="Number
149     of hex rows")
150 parser.add_argument("--cols", type=int, default=12, help="Number
151     of hex columns")
152 parser.add_argument("--radius", type=float, default=1.0, help="
153     Hexagon radius R")
154 parser.add_argument(
155     "--output",
156     type=str,
157     default=None,
158     help="Save figure to this path instead of only displaying",
159 )
160 parser.add_argument(
161     "--no-show",
162     action="store_true",
163     help="Do not open an interactive plot window",
164 )
165 parser.add_argument(
166     "--proof",
167     action="store_true",
168     help="Also print the  $D = \sqrt{R(3N)}$  proof and draw the geometry
169     diagram",
170 )
171 parser.add_argument(
172     "--proof-output",
173     type=str,
174     default=None,
175     help="Save proof diagram to this path (used with --proof)",
176 )
177 return parser.parse_args(argv)
178
179
180 def prompt_positive_int(name: str) -> int:
181     while True:
182         raw = input(f"Enter {name}: ").strip()
183         try:
184             value = int(raw)
185         except ValueError:
186             print(f"    Please enter a valid integer for {name}.")
187             continue
188         if value < 0:
189             print(f"    {name} must be non-negative.")
190             continue
191         return value

```



```

185
186
187 def main(argv: list[str] | None = None) -> int:
188     args = parse_args(argv)
189
190     if args.i is not None and args.j is not None:
191         i, j = args.i, args.j
192     elif args.i is None and args.j is None:
193         i = prompt_positive_int("i")
194         j = prompt_positive_int("j")
195     else:
196         print("Error: provide both -i and -j, or neither for
197             interactive input.")
198         return 1
199
200     if i == 0 and j == 0:
201         print("Error: i and j cannot both be zero (N would be 0).")
202         return 1
203
204     gcd_ij = math.gcd(i, j)
205     if gcd_ij > 1:
206         print(
207             f"Note: gcd(i, j) = {gcd_ij}. For a full N-cell reuse
208             pattern, "
209             f"i and j are usually coprime (e.g. i=2, j=1).")
210
211     if args.proof:
212         from hexagonal_proof import plot_proof_diagram, print_proof
213
214         print()
215         print_proof(i, j, args.radius)
216         plot_proof_diagram(
217             i,
218             j,
219             args.radius,
220             output_path=args.proof_output,
221             show=not args.no_show,
222         )
223
224     plot_hexagonal_layout(
225         i=i,
226         j=j,
227         rows=args.rows,
228         cols=args.cols,
229         radius=args.radius,
230         output_path=args.output,
231         show=not args.no_show,
232     )
233     return 0

```

```

234
235 if __name__ == "__main__":
236     sys.exit(main())

```

Listing 2.1: Complete Python implementation for hexagonal cluster allocation.

2.4 Program Output

The program was executed for three representative (i, j) pairs. Each figure shows the cluster number inside every hexagon and uses a distinct color per cluster.

2.4.1 Case 1: $i = 1, j = 1, N = 3$

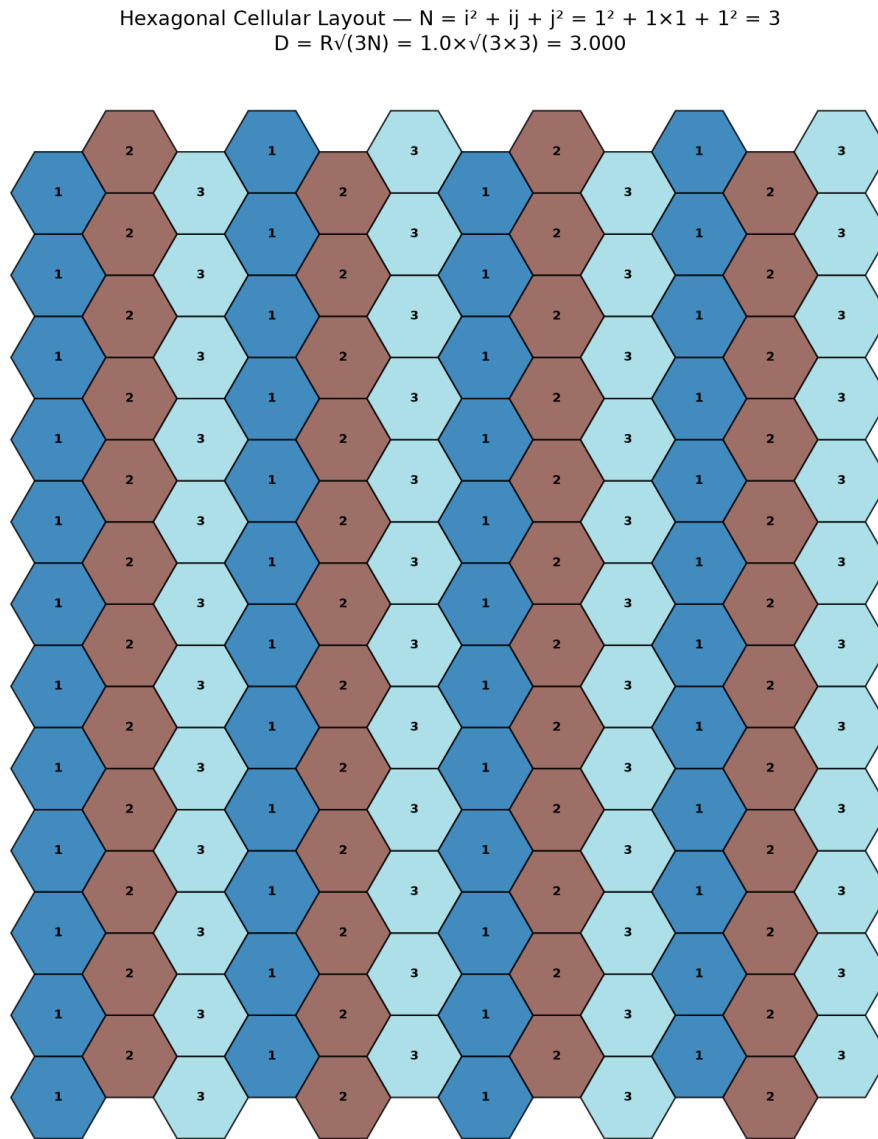


Figure 2.1: Hexagonal layout for $i = 1, j = 1$. Cluster size $N = 1^2 + 1 \cdot 1 + 1^2 = 3$.

2.4.2 Case 2: $i = 2, j = 1, N = 7$

Hexagonal Cellular Layout — $N = i^2 + ij + j^2 = 2^2 + 2 \times 1 + 1^2 = 7$
 $D = R\sqrt{3N} = 1.0 \times \sqrt{3 \times 7} = 4.583$

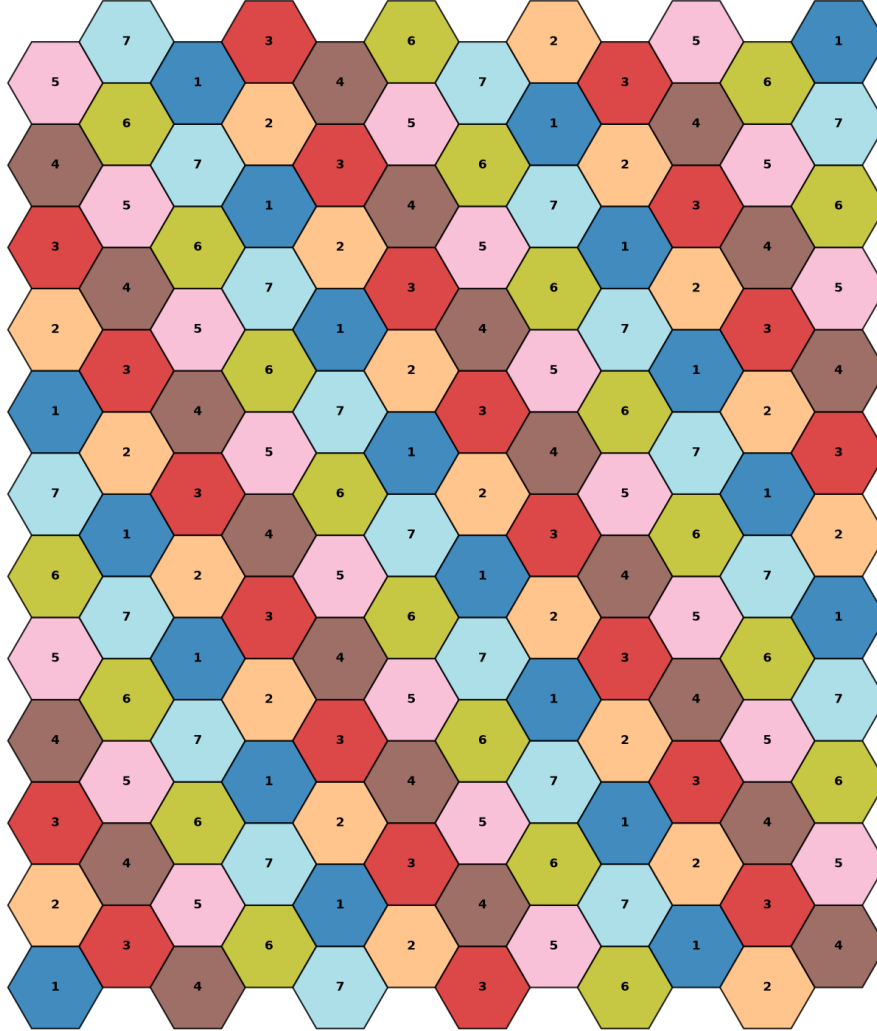


Figure 2.2: Hexagonal layout for $i = 2, j = 1$. Cluster size $N = 2^2 + 2 \cdot 1 + 1^2 = 7$. Reuse distance $D = R\sqrt{3N} = \sqrt{21} \approx 4.583$.

2.4.3 Case 3: $i = 3, j = 2, N = 19$

$$\text{Hexagonal Cellular Layout} \rightarrow N = i^2 + ij + j^2 = 3^2 + 3 \times 2 + 2^2 = 19$$

$$D = R\sqrt{3N} = 1.0 \times \sqrt{3 \times 19} = 7.550$$

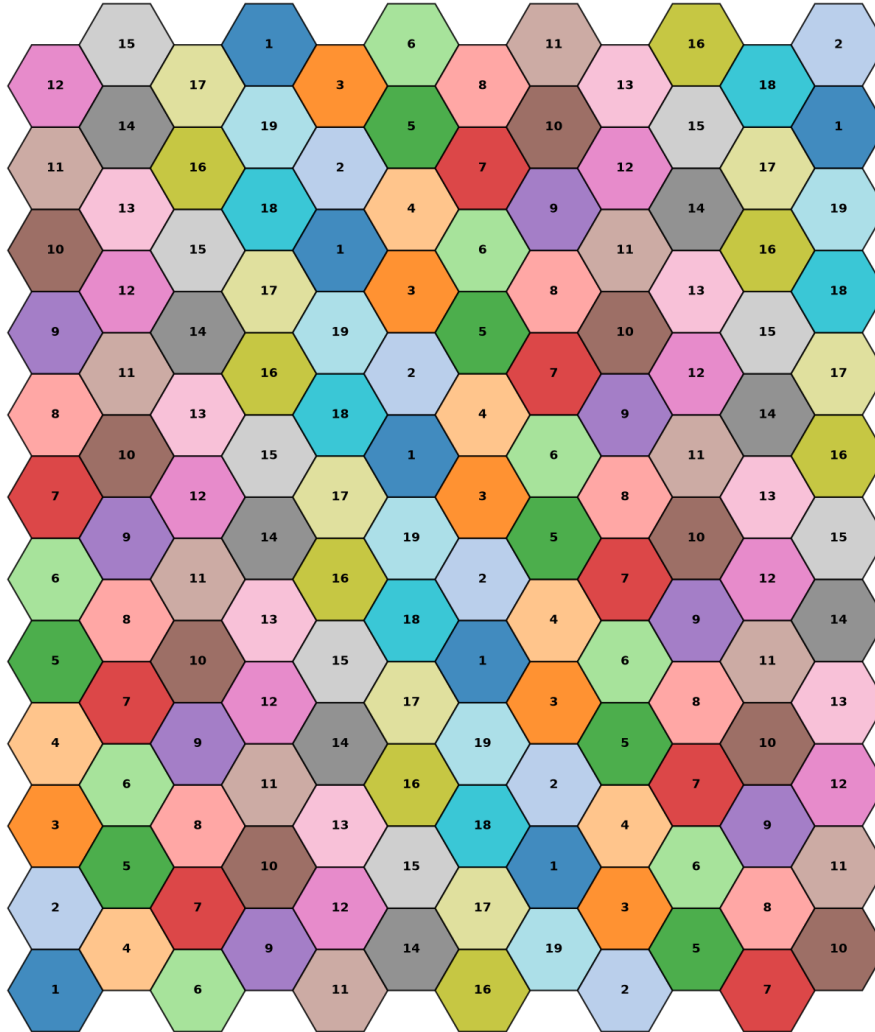


Figure 2.3: Hexagonal layout for $i = 3, j = 2$. Cluster size $N = 3^2 + 3 \cdot 2 + 2^2 = 19$.

2.5 Sample Console Output

For $i = 2, j = 1$, and $R = 1.0$, the program prints:

```
Cluster parameters: i = 2, j = 1
Cluster size (N)      =  $i^2 + ij + j^2 = 2^2 + 2 \times 1 + 1^2 = 7$ 
Cell radius (R)      = 1.0
Reuse distance (D)    =  $R\sqrt{3N} = 4.5826$ 
Reuse ratio (Q=D/R)  =  $\sqrt{3N} = 4.5826$ 
```

Chapter 3

Proof: Co-channel Reuse Distance in Hexagonal Geometry

3.1 Statement

To prove: For hexagonal cellular geometry,

$$D = R\sqrt{3N}, \quad Q = \frac{D}{R} = \sqrt{3N} \quad (3.1)$$

where

- R = radius of a regular hexagonal cell (center to vertex),
- $N = i^2 + ij + j^2$ = cluster size,
- i, j = non-negative integers along two reuse directions,
- D = distance between centers of two co-channel cells.

3.2 Geometric Setup

Refer to Figure 3.1. Starting from co-channel cell center **A**:

1. move i hex steps along adjacent cell centers to reach **B**;
2. turn through the hex-grid direction and move j steps to reach **C**.

The co-channel reuse distance is $D = AC$. Let R_p denote half the center-to-center spacing between adjacent cells. Each step has length $2R_p$, so:

$$AB = i \cdot 2R_p \quad (3.2)$$

$$BC = j \cdot 2R_p \quad (3.3)$$

$$\angle ABC = 120^\circ \quad (\cos 120^\circ = -\frac{1}{2}) \quad (3.4)$$

Hexagonal Reuse Geometry — $N = 2^2 + 2 \times 1 + 1^2 = 7$, $D = R\sqrt{3N} = 1.0 \times \sqrt{3 \times 7} = 4.583$

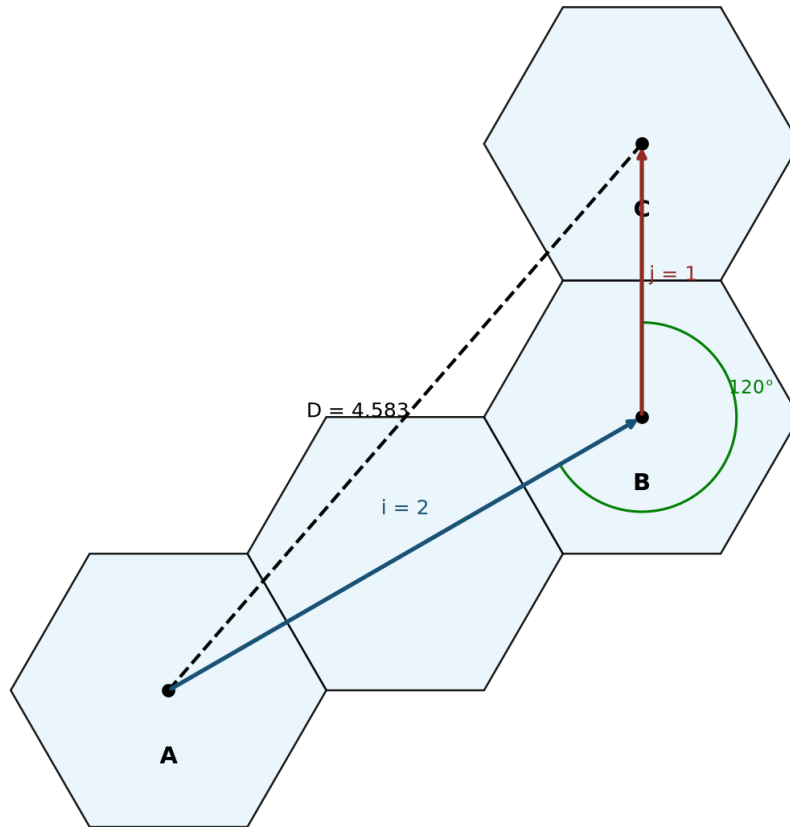


Figure 3.1: Reuse geometry for $i = 2$, $j = 1$. Points A and C are co-channel cell centers separated by distance D .

3.3 Derivation

3.3.1 Step 1: Law of Cosines

Applying the law of cosines in $\triangle ABC$:

$$D^2 = (i \cdot 2R_p)^2 + (j \cdot 2R_p)^2 - 2(i \cdot 2R_p)(j \cdot 2R_p) \cos(120^\circ) \quad (3.5)$$

Substituting $\cos(120^\circ) = -\frac{1}{2}$:

$$D^2 = 4R_p^2 i^2 + 4R_p^2 j^2 - 2 \cdot 4R_p^2 ij \left(-\frac{1}{2}\right) \quad (3.6)$$

$$D^2 = 4R_p^2 (i^2 + ij + j^2) \quad (3.7)$$

3.3.2 Step 2: Relation Between R_p and R

For a regular hexagon of radius R :

$$R_p = \frac{\sqrt{3}}{2} R \quad \implies \quad 2R_p = \sqrt{3} R \quad (3.8)$$

Substituting into (3.7):

$$D^2 = 4 \cdot \frac{3R^2}{4} \cdot (i^2 + ij + j^2) \quad (3.9)$$

$$D^2 = 3R^2(i^2 + ij + j^2) \quad (3.10)$$

3.3.3 Step 3: Cluster Size

By definition of cluster size in hexagonal reuse:

$$N = i^2 + ij + j^2 \quad (3.11)$$

Therefore:

$$D = R\sqrt{3N} \quad (3.12)$$

The co-channel reuse ratio is:

$$Q = \frac{D}{R} = \sqrt{3N} \quad (3.13)$$

3.4 Numerical Verification ($i = 2, j = 1, R = 1$)

Table 3.1: Numerical verification of the reuse distance formula.

Quantity	Value
$N = i^2 + ij + j^2$	$2^2 + 2(1) + 1^2 = 7$
$R_p = \frac{\sqrt{3}}{2}R$	0.8660
$2R_p = \sqrt{3}R$	1.7321
D (law of cosines)	$\sqrt{21} = 4.5826$
$D = R\sqrt{3N}$	$\sqrt{3 \times 7} = 4.5826$
$Q = D/R = \sqrt{3N}$	4.5826

Both methods yield identical results, confirming the proof.

3.5 Supporting Code: hexagonal_proof.py

The proof module verifies the derivation numerically and generates the geometry diagram shown in Figure 3.1.

```

1  """
2  Proof: For hexagonal cellular geometry,  $D = \sqrt{R(3N)}$ , where  $N = i^2 + ij + j^2$ .
3
4  This follows the assignment derivation using the law of cosines on
   the
5   $(i, j)$  reuse vector with adjacent cell spacing  $2R_p$  and  $R_p = \sqrt{(3/2)R}$ .
6  """

```

```

7
8 from __future__ import annotations
9
10 import argparse
11 import math
12 import sys
13
14 import matplotlib.pyplot as plt
15 import numpy as np
16 from matplotlib.patches import RegularPolygon
17
18
19 def cluster_size(i: int, j: int) -> int:
20     return i * i + i * j + j * j
21
22
23 def adjacent_center_distance(radius: float) -> float:
24     """Center-to-center distance between two adjacent hexagonal
25         cells."""
26     rp = (math.sqrt(3) / 2) * radius
27     return 2 * rp
28
29 def axial_to_cartesian(q: int, r: int, radius: float) -> tuple[float, float]:
30     """Convert axial hex coordinates to Cartesian (pointy-top layout
31         )."""
32     x = 1.5 * radius * q
33     y = math.sqrt(3) * radius * (r + q / 2)
34     return x, y
35
36 def reuse_distance_formula(radius: float, n: int) -> float:
37     return radius * math.sqrt(3 * n)
38
39
40 def reuse_ratio(n: int) -> float:
41     return math.sqrt(3 * n)
42
43
44 def print_proof(i: int, j: int, radius: float = 1.0) -> None:
45     """Print the full step-by-step proof from the assignment."""
46     n = cluster_size(i, j)
47     rp = (math.sqrt(3) / 2) * radius
48     two_rp = 2 * rp
49     cos_120 = math.cos(math.radians(120))
50     d_squared = (i * two_rp) ** 2 + (j * two_rp) ** 2 - 2 * (i *
51         two_rp) * (j * two_rp) * cos_120
52     d_from_steps = math.sqrt(d_squared)
53     d_from_formula = reuse_distance_formula(radius, n)
54     q_ratio = reuse_ratio(n)

```



```

54  bar = "=" * 72
55
56
57  print(bar)
58  print("PROOF: Co-channel reuse distance in hexagonal geometry")
59  print(bar)
60  print()
61  print("To prove:")
62  print("      D =  $\sqrt{3}R$       and      Q = D/R =  $\sqrt{3}$ ")
63  print("where")
64  print("      R = radius of a regular hexagonal cell")
65  print("      N =  $i^2 + ij + j^2$  (cluster size)")
66  print("      i, j = non-negative integers along two 120° reuse directions")
67  print("      D = distance between centers of two co-channel cells")
68  print()
69  print("-" * 72)
70  print("STEP 1 - Geometry of the reuse vector")
71  print("-" * 72)
72  print()
73  print("Refer to the diagram: move i hex steps from A to B, then j steps")
74  print("from B to C along a direction 120° to AB. The co-channel distance is AC = D.")
75  print()
76  print("Let Rp = center-to-center distance of adjacent hexagonal cells / 2")
77  print("      (half the spacing between neighboring cell centers).")
78  print()
79  print("Each step spans 2Rp, so:")
80  print(f"      AB = i · 2Rp = {i} × {two_rp:.4f} = {i * two_rp:.4f}")
81  print(f"      BC = j · 2Rp = {j} × {two_rp:.4f} = {j * two_rp:.4f}")
82  print(f"      ABC = 120° (cos 120° = {cos_120:.1f})")
83  print()
84  print("-" * 72)
85  print("STEP 2 - Law of cosines")
86  print("-" * 72)
87  print()
88  print("Applying the law of cosines in triangle ABC (interior angle at B = 120°):")
89  print()
90  print("      D2 = (i·2Rp)2 + (j·2Rp)2 - 2(i·2Rp)(j·2Rp) cos(120°)")
91  print()
92  print(f"Substituting cos(120°) = {cos_120:.1f}:")
93  print()
94  print("      D2 = 4Rp2i2 + 4Rp2j2 - 2·4Rp2ij·-(½)")
95  print("      D2 = 4Rp2 (i2 + ij + j2)")

```

```

96     print()
97     print(f"Numerical check with i={i}, j={j}, R={radius}:")
98     print(f"      D2 = {d_squared:.6f} → D = {d_from_steps:.6f}")
99     print()
100    print("-" * 72)
101    print("STEP 3 - Relating Rp to cell radius R")
102    print("-" * 72)
103    print()
104    print("For a regular hexagon of radius R (center to vertex):")
105    print()
106    print("      Rp =  $\sqrt{3 / 2} \cdot R$ ")
107    print()
108    print(f"With R = {radius}:")
109    print(f"      Rp = {rp:.6f}")
110    print(f"      2Rp =  $\sqrt{3} \cdot R = {two_rp:.6f}$ ")
111    print()
112    print("Substituting Rp into D2:")
113    print()
114    print("      D2 = 4 · (3R2/4) · (i2 + ij + j2)")
115    print("      D2 = 3R2 · (i2 + ij + j2)")
116    print()
117    print("-" * 72)
118    print("STEP 4 - Cluster size N")
119    print("-" * 72)
120    print()
121    print("By definition of cluster size in hexagonal reuse:")
122    print()
123    print(f"      N = i2 + ij + j2 = {i}2 + {i}×{j} + {j}2 = {n}")
124    print()
125    print("Therefore:")
126    print()
127    print("      D2 = 3R2N")
128    print("      D =  $\sqrt{R(3N)}$ ")
129    print()
130    print("Co-channel reuse ratio:")
131    print()
132    print("      Q = D/R =  $\sqrt{3N}$ ")
133    print()
134    print("-" * 72)
135    print("STEP 5 - Numerical verification")
136    print("-" * 72)
137    print()
138    print(f"Given: i = {i}, j = {j}, R = {radius}")
139    print(f"      N = {n}")
140    print(f"      D (from law of cosines) = {d_from_steps:.6f}")
141    print(f"      D (from formula  $\sqrt{R(3N)}$ ) = {d_from_formula:.6f}")
142    print(f"      Q = D/R = {q_ratio:.6f}")
143    print(f"       $\sqrt{3N}$  = {q_ratio:.6f}")
144    print()
145
146    if math.isclose(d_from_steps, d_from_formula, rel_tol=1e-9):

```

```

147     print(" Both methods agree - proof verified numerically.")
148 else:
149     print(" Numerical mismatch - check parameters.")
150
151 print()
152 print(" D =  $\sqrt{R(3N)}$  proved.")
153 print(bar)
154
155
156 def draw_hex_at(ax, q: int, r: int, radius: float, **kwargs) ->
    tuple[float, float]:
157     x, y = axial_to_cartesian(q, r, radius)
158     defaults = dict(
159         numVertices=6,
160         radius=radius,
161         orientation=np.radians(30),
162         edgecolor="black",
163         linewidth=1.2,
164         facecolor="#e8f4fc",
165         alpha=0.9,
166     )
167     defaults.update(kwargs)
168     ax.add_patch(RegularPolygon((x, y), **defaults))
169     return x, y
170
171
172 def plot_proof_diagram(
173     i: int,
174     j: int,
175     radius: float = 1.0,
176     output_path: str | None = None,
177     show: bool = True,
178 ) -> None:
179     """Draw the --ABC reuse-vector diagram from the assignment."""
180     n = cluster_size(i, j)
181     d_formula = reuse_distance_formula(radius, n)
182
183     ax_x, ax_y = axial_to_cartesian(0, 0, radius)
184     bx, by = axial_to_cartesian(i, 0, radius)
185     cx, cy = axial_to_cartesian(i, j, radius)
186
187     # Draw hex cells along the path A → B → C and at endpoints.
188     cells = {(0, 0), (i, 0), (i, j)}
189     for step in range(1, i):
190         cells.add((step, 0))
191     for step in range(1, j):
192         cells.add((i, step))
193
194     fig, ax = plt.subplots(figsize=(10, 8))
195
196     for q, r in cells:

```

```

197     draw_hex_at(ax, q, r, radius)
198
199     # Highlight centers A, B, C.
200     for label, (x, y) in zip("ABC", [(ax_x, ax_y), (bx, by), (cx, cy)
201                                     ]]):
202         ax.plot(x, y, "ko", markersize=7)
203         ax.text(x, y - 0.35 * radius, label, ha="center", va="top",
204                 fontsize=14, fontweight="bold")
205
206     # Vectors i and j along AB and BC.
207     ax.annotate(
208         "",
209         xy=(bx, by),
210         xytext=(ax_x, ax_y),
211         arrowprops=dict(arrowstyle="->", color="#1a5276", lw=2.5),
212     )
213     ax.text((ax_x + bx) / 2, (ax_y + by) / 2 + 0.25 * radius, f"i = {i}",
214             ha="center", fontsize=12, color="#1a5276")
215
216     ax.annotate(
217         "",
218         xy=(cx, cy),
219         xytext=(bx, by),
220         arrowprops=dict(arrowstyle="->", color="#922b21", lw=2.5),
221     )
222     ax.text((bx + cx) / 2 + 0.2 * radius, (by + cy) / 2, f"j = {j}",
223             ha="center", fontsize=12, color="#922b21")
224
225     # Co-channel distance D (dashed).
226     ax.plot([ax_x, cx], [ax_y, cy], "k--", lw=2)
227     ax.text((ax_x + cx) / 2 - 0.3 * radius, (ax_y + cy) / 2, f"D = {
228             d_formula:.3f}", ha="center", fontsize=12)
229
230     # 120° angle arc at B.
231     v1 = np.array([ax_x - bx, ax_y - by])
232     v2 = np.array([cx - bx, cy - by])
233     angle1 = math.degrees(math.atan2(v1[1], v1[0]))
234     angle2 = math.degrees(math.atan2(v2[1], v2[0]))
235     arc = plt.matplotlib.patches.Arc(
236         (bx, by),
237         width=1.2 * radius,
238         height=1.2 * radius,
239         angle=0,
240         theta1=min(angle1, angle2),
241         theta2=max(angle1, angle2),
242         color="green",
243         lw=1.5,
244     )
245     ax.add_patch(arc)
246     ax.text(bx + 0.55 * radius, by + 0.15 * radius, "120°", fontsize
247             =11, color="green")

```

```

242     ax.set_aspect("equal")
243     ax.autoscale_view()
244     ax.set_title(
245         f"Hexagonal Reuse Geometry -  $N = i^2 + i \times j + j^2 = n$ ,
246         "
247         f" $D = \sqrt{R(3N)} = \text{radius} \times \sqrt{3 \times n} = \text{d\_formula:.3f}$ ",
248         fontsize=12,
249     )
250     plt.axis("off")
251     plt.tight_layout()
252
253     if output_path:
254         fig.savefig(output_path, dpi=150, bbox_inches="tight")
255         print(f"Saved proof diagram to: {output_path}")
256
257     if show:
258         plt.show()
259     else:
260         plt.close(fig)
261
262
263 def parse_args(argv: list[str] | None = None) -> argparse.Namespace:
264     parser = argparse.ArgumentParser(
265         description="Prove and verify  $D = \sqrt{R(3N)}$  for hexagonal
266         cellular geometry."
267     )
268     parser.add_argument("-i", "--i-param", type=int, dest="i",
269                         default=2, help="Reuse parameter i")
270     parser.add_argument("-j", "--j-param", type=int, dest="j",
271                         default=1, help="Reuse parameter j")
272     parser.add_argument("--radius", type=float, default=1.0, help="
273         Cell radius R")
274     parser.add_argument(
275         "--diagram",
276         type=str,
277         default=None,
278         help="Save the A-B-C geometry diagram to this path",
279     )
280     parser.add_argument("--no-show", action="store_true", help="Do
281         not open plot window")
282     return parser.parse_args(argv)
283
284
285 def main(argv: list[str] | None = None) -> int:
286     args = parse_args(argv)
287
288     if args.i == 0 and args.j == 0:
289         print("Error: i and j cannot both be zero.")
290         return 1

```

```
287     print_proof(args.i, args.j, args.radius)
288     plot_proof_diagram(
289         args.i,
290         args.j,
291         args.radius,
292         output_path=args.diagram,
293         show=not args.no_show,
294     )
295     return 0
296
297
298 if __name__ == "__main__":
299     sys.exit(main())
```

Listing 3.1: Python module for proof verification and geometry diagram generation.

Chapter 4

Conclusion

This assignment successfully demonstrates hexagonal frequency reuse in cellular systems. The Python program tessellates a workspace into regular hexagons and assigns cluster IDs using $N = i^2 + ij + j^2$. The mathematical proof establishes that the co-channel reuse distance is $D = R\sqrt{3N}$, which was verified both analytically and numerically for $i = 2$, $j = 1$.

— *End of Report* —